5 Aggregatfunktionen

- Anzahl aller Hotels

```
SELECT COUNT(*)
FROM hotel;
```

```
SELECT COUNT(ort)
FROM hotel;
```

- Anzahl der verschiedenen Orte

```
SELECT COUNT(DISTINCT ort)
FROM hotel;
```

- Maximaler Zimmerpreis über alle Hotels

```
SELECT MAX(preis)
FROM zimmer;
```

- Maximaler Zimmerpreis für jedes Hotel

```
SELECT MAX(preis), honr
FROM zimmer
GROUP BY honr;
```

5 Aggregatfunktionen

- Auflistung der Hotelnamen mit teuerstem Zimmer

```
SELECT h.honr, h.hname
FROM zimmer z JOIN hotel h USING (honr)
WHERE z.preis IN (
          SELECT MAX(preis)
          FROM zimmer);
```

- Auflistung der teuersten Zimmer eines jeden Hotels

```
SELECT h.honr, h.hname, z.znr, z.zname, z.preis
FROM zimmer z JOIN hotel h USING (honr)
WHERE (z.honr, z.preis) IN (
          SELECT z.honr, MAX(preis)
          FROM zimmer z
          GROUP BY z.honr);
```

5 Eigenschaftsüberprüfung von Aggregatfunktionen

- Auflistung aller Hotels, deren maximaler Zimmerpreis kleiner als 150 Euro ist

```
SELECT h.honr, h.hname
FROM hotel h JOIN zimmer z USING (honr)
GROUP BY h.honr, h.hname
HAVING MAX(z.preis) < 150;
```

- Bedingung ist über Gruppe von Zimmern (des jeweiligen Hotels).
- Solche Bedingungen müssen hinter having erfolgen.

5 Aggregatfunktionen

- Auflistung des Gesamtumsatzes eines Kunden (nur Zimmerreservierungen)

```
SELECT g.gnr, g.name, SUM(z.preis*(rp.bis-rp.von)) AS umsatz
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN zimmer z ON z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name;
```



Bei der Verwendung von Aggregatfunktionen müssen alle Attribute, auf die im SELECT-Teil projiziert wird und die **nicht** in der Aggregatfunktion verwendet werden, in der GROUP BY-Klausel aufgeführt werden.

5 UNION

- Auflistung des Gesamtumsatzes von allen Kunden (nur Zimmerreservierungen)

```
SELECT g.gnr, g.name, SUM(z.preis*(rp.bis-rp.von)) AS umsatz
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN zimmer z ON z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name;
```

- Sind dies alle Kunden? Jetzt schon:

```
SELECT g.gnr, g.name, SUM(z.preis*(rp.bis-rp.von)) AS umsatz
FROM gast g LEFT JOIN reservierung r USING (gnr)
            LEFT JOIN rposition rp USING (rnr)
            LEFT JOIN zimmer z ON
                      z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name;
```

- Problem: Kunden mit Umsatz 0! Dort steht NULL.

5 UNION

- Problem: Möglicherweise Anzeige NULL statt 0

```
SELECT g.gnr, g.name, SUM(z.preis*(rp.bis-rp.von)) AS umsatz
FROM gast g JOIN reservierung r USING (gnr)
            JOIN rposition rp USING (rnr)
            JOIN zimmer z ON z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name
UNION
SELECT g.gnr, g.name, 0
FROM gast g
WHERE g.gnr NOT IN (SELECT r.gnr FROM reservierung r);
```

- Aber: UNION ist langsam!

5 Bedingte Ausgaben

- Ausgabewerte im SELECT können an Bedingungen geknüpft werden

```
CASE  {<ausdruck> | <spaltenname>}
WHEN  <wert>
THEN  {<wert> | < ausdruck > | <spaltenname>}
…
ELSE  {<wert> | < ausdruck > | <spaltenname>}
END
```

5 Bedingte Ausgaben

- Auflistung des Gesamtumsatzes eines Kunden mit LEFT JOIN und Ersetzung des NULL-Wertes durch 0

```
SELECT g.gnr, g.name, CASE (SUM(z.preis*(rp.bis-rp.von)) IS NULL)
                      WHEN true
                      THEN 0
                      ELSE SUM(z.preis*(rp.bis-rp.von)) END
                      AS umsatz
FROM gast g LEFT JOIN reservierung r USING (gnr)
            LEFT JOIN rposition rp USING (rnr)
            LEFT JOIN zimmer z ON
                      z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name;
```

5 View Definition Language

- View ist eine virtuelle Tabelle ohne eigene Datensätze
  - Daten der View aktualisieren sich automatisch, wenn benutzte Tabellen zur Definition der View sich ändern
  - Datenretrieval wie normale Tabelle
  - Datenspeicherung nicht oder nur begrenzt möglich
- Erstellung individueller Sichten auf die Datenbasis
  - Gezielte Denormalisierung zur Datenaufbereitung
  - Zugriffssteuerung
  - Vereinfachung komplexer Abfragen / „Speicherung“ von Zwischenergebnissen
  - Stabile Schnittstelle in andere Systeme

5 View Definition Language

```
CREATE VIEW <viewname> [(<spaltenname> [,  ...])] AS <select befehl>;
```

```
REPLACE VIEW <viewname> [(<spaltenname> [,  ...])] AS <select befehl>;
```

```
CREATE OR REPLACE VIEW <viewname> [(<spaltenname> [,  ...])]
                                                  AS <select befehl>;
```

```
DROP VIEW <viewname>;
```

5 Erstellung einer individuellen Sicht / Datenaufbereitung

- Erstellung der Kundenauswertung als VIEW

```
CREATE VIEW kundenumsatz AS
SELECT g.gnr, g.name, CASE (SUM(z.preis*(rp.bis-rp.von)) IS NULL)
                      WHEN true
                      THEN 0
                      ELSE SUM(z.preis*(rp.bis-rp.von)) END
                      AS umsatz
FROM gast g LEFT JOIN reservierung r USING (gnr)
            LEFT JOIN rposition rp USING (rnr)
            LEFT JOIN zimmer z ON
                      z.honr = rp.honr AND z.znr = rp.znr
GROUP BY g.gnr, g.name;
```

5 View als Zwischenspeicher

- Für welches Hotel wurde am häufigsten reserviert (nur Anzahl Reservierungen)?
  - 1. Schritt: Bestimmung der Anzahl der Reservierungen pro Hotel

```
CREATE VIEW anzahl_res (honr, anzahl) AS
SELECT rp.honr, COUNT(*)
FROM rposition rp
GROUP BY rp.honr;
```

  - 2. Schritt: Selektion des Hotels mit der maximalen Anzahl

```
SELECT a.honr, a.anzahl, h.hname
FROM anzahl_res a JOIN hotel h USING (honr)
WHERE a.anzahl IN (SELECT MAX(anzahl) FROM anzahl_res);
```

5 Indizierung

- Erstellung eines Suchindexes zum schnelleren Auffinden von Datensätzen. Wichtig für Fremdschlüssel.

```
CREATE INDEX <indexname> ON <tabellenname> (spaltenname [,...]) ;
```

```
DROP INDEX <indexname>;
```

- Beispiel: Indizierung des Hotelnamens

```
CREATE INDEX ihotelname ON hotel (hname);
```

Bemerkungen zum Index:

- Kann auf mehrere Spalten angewendet werden.
- Erhöht den Speicherplatz, verlangsamt Updates
- Suchzeit **sinkt** von linear auf logarithmisch oder konstant!

5 Vergabe von Zugriffsrechten

```
GRANT {ALL PRIVILEGES | CREATE TABLE | ALTER | DROP | SELECT |
       INSERT | UPDATE | DELETE | EXECUTE} [, …]
ON <objektname>
TO {<username> | PUBLIC | <rollenname>}
[WITH GRANT OPTION];
```

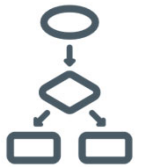
```
REVOKE {CREATE TABLE | ALTER | DROP | SELECT | INSERT | UPDATE |
        DELETE | EXECUTE} [, …]
ON <objektname>
FROM {<username> | PUBLIC | <rollenname>};
```

```
CREATE USER <username> IDENTIFIED BY Password('<password>');
```

```
CREATE ROLE <rollenname>;                    DROP ROLE <rollenname>;
```

- Objekt: Tabelle, View, Stored Procedure
- Grantoption: Weitergabe der Rechte
  Achtung: Revoke nicht kaskadierende Rechte-Zurücknahme!

5 Stored Procedures / Stored Functions



Stored Procedure
In Datenbank gespeicherte, vorkompilierte Routine, die vordefinierte Datenbankoperationen gemäß einer Ablaufsteuerung ausführt

- Nicht von allen DBMS unterstützt
- Stark plattformabhängig

5 Stored Procedures / Stored Functions

- Auslagerung oft verwendeter Befehlsfolgen in das DBMS
- Reduktion des Netzwerkverkehrs
- Erhöhung der Sicherheit durch Vermeidung direkter Zugriffe auf Datenbanktabellen
- Plattformunabhängig
- Wiederverwendbar in mehreren Programmen, da in DB implementiert und nicht im Programmcode
- Laufzeitverringerung durch Vorkompilierung
- Erweiterungen in Postgres: plpgsql

https://www.postgresql.org/docs/15/plpgsql.html

5 Stored Procedures – Syntax PostgreSQL

```
CREATE PROCEDURE <procedurename> (
   [[{IN | OUT | INOUT}] <parametername> DATENTYP][, …]
)
LANGUAGE SQL | PLPGSQL
BEGIN ATOMIC | AS $$
BEGIN
   <anweisung>;
      …
END; | END; $$
```

- IN: Ausschließlich Eingabeparameter (Default)
- OUT: Ausschließlich Ausgabeparameter
- INOUT: Sowohl Eingabe- wie auch Ausgabeparameter

- PLPGSQL: Erweiterte Funktionalität von SQL durch bedingte Anweisungen, Schleifen, Variablen und andere prozedurale Konstrukte
- BEGIN ATOMIC: Alles in der Prozedur wird ausgeführt, ohne dass andere Benutzer dazwischen SQL-Befehle ausführen

5 Stored Procedures – Trennzeichen

Problem

- Standardmäßiges Trennzeichen (Delimiter) von SQL-Befehlen ist das Semikolon
- Trennzeichen bei Stored Procedures ist ebenfalls das Semikolon

⇒ Temporärer Wechsel des Trennzeichens

Mit **`LANGUAGE PLPGSQL AS $$ … END; $$`** (Folie zuvor)

Oder:

```
DELIMITER //  oder SET TERM //
CREATE PROCEDURE hotel_suche_name (IN name CHAR(50))
BEGIN
  SELECT * FROM hotel WHERE hname = name;
END //
DELIMITER ;   oder SET TERM ;
```

5 Stored Procedures

- Löschen:

```
DROP PROCEDURE <procedurename>;
```

- Aufruf:

```
CALL | EXEC | EXECUTE <procedurename> ();
```

- Einfaches Beispiel:

```
CREATE PROCEDURE hotel_suche_name (IN name CHAR(50))
BEGIN
  SELECT * FROM hotel WHERE hname = name;
END
```

5 Stored Procedures – Variablen & Anweisungen

- Variablendeklaration

```
DECLARE <variablenname> DATENTYP [DEFAULT WERT];
```

- Zuweisung PostGreSQL

```
SET <variablenname> := {<variablenname> | <ausdruck>};
```

- Zuweisung MariaDB

```
SET <variablenname> = {<variablenname> | <ausdruck>};
```

- Ausgabe MariaDB und eingeschränkt PostGreSQL

```
SELECT <ausgabe>;
```

Stored Procedures – Kontrollstrukturen

- Verzweigung

```
IF <bedingung>
  THEN
  <anweisung>;
   …
[[ELSE IF <bedingung>
  THEN
   <anweisung>;
     …]
 [ELSE
   <anweisung>;
     …]]
END IF;
```

Stored Procedures – Kontrollstrukturen

- Simple Case

```
CASE <variable>
  WHEN WERT THEN <anweisung>; …
  …
 [ELSE <anweisung>; …]
END CASE;
```

- Searched Case

```
CASE
  WHEN <bedingung> THEN <anweisung>; …
  …
 [ELSE <anweisung>; …]
END CASE;
```

Stored Procedures – Kontrollstrukturen

- LOOP

```
<loopname>: LOOP
    EXIT WHEN <bedingung>; |
    IF <bedingung> THEN EXIT <loopname>; END IF; |
<anweisung>; …
END LOOP <loopname>;
```

- WHILE

```
WHILE <bedingung> LOOP
    <anweisung>; …
END LOOP;
```

- FOR

```
FOR <zähler> IN [REVERSE] <Start> .. <Ende> [BY <Inkrement>] LOOP
 <anweisung>; …
END LOOP;
```

5 Stored Procedures – Datensätze auslesen

- Variante 1

```
SELECT <attributname> [, …] INTO <variablenname> [, …]
FROM … ;
```

  - Anzahl Attribute und Anzahl Variablen müssen gleich sein
  - Select-Befehl darf nur einen Datensatz zurückgeben

- Variante 2

```
SELECT @<variablenname> := <attributname> [, …]
FROM … ;
```

  - Variable erhält den letzten Datensatz des Select-Befehls

5 Stored Procedures – Datensätze auslesen

- Problem: Select liefert i.d.R. mehrere Datensätze

⇒ CURSOR

```
DECLARE <cursorname> CURSOR FOR SELECT …;
```

```
OPEN <cursorname>;
```

```
CLOSE <cursorname>;
```

- Zugriff auf nächsten Datensatz

```
FETCH <cursorname> INTO <variablenname> [, …];
```

- Herausfinden, ob noch Datensatz vorhanden

```
IF FOUND THEN
```

5 Stored Procedures – Beispiel PostgreSQL

```
CREATE PROCEDURE gaestesichern ()
LANGUAGE plpgsql
AS $$
DECLARE
  gastname CHAR(50);
  gastnr INTEGER;
  finished BOOLEAN DEFAULT FALSE;
  gastcursor CURSOR FOR SELECT name, gnr FROM gast;
BEGIN
  OPEN gastcursor;
  WHILE NOT finished LOOP
    FETCH gastcursor INTO gastname, gastnr;
    IF FOUND THEN
       INSERT INTO altgaeste (gnr, gname)
                    VALUES (gastnr, gastname);
    ELSE finished := TRUE;
    END IF;
  END LOOP;
  CLOSE gastcursor;
END $$;
```

5 Stored Procedures – Beispiel MySQL

```
DELIMITER //
CREATE PROCEDURE gaestesichern ()
BEGIN
  DECLARE gastname CHAR(50);
  DECLARE gastnr INTEGER;
  DECLARE finished BOOLEAN DEFAULT FALSE;
  DECLARE gastcursor CURSOR FOR SELECT name, gnr FROM gast;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET finished = TRUE;

  OPEN gastcursor;
  REPEAT
    FETCH gastcursor INTO gastname, gastnr;
    IF NOT finished THEN
       INSERT INTO altgaeste (gnr, gname)
                   VALUES (gastnr, gastname);
    END IF;
  UNTIL finished END REPEAT;
  CLOSE gastcursor;
END //
DELIMITER ;
```

5 Stored Procedures – Erklärung Folie zuvor

- Problem:
  Nach letztem Datensatz wird bei FETCH ein Fehler geworfen

⇒ Error HANDLER

```
DECLARE CONTINUE HANDLER FOR {NOT FOUND | SQLSTATE '02000'}
<anweisung>;
```

5 Stored Functions

```
CREATE FUNCTION <functionname> (
   [[{IN | OUT | INOUT}] <parametername> DATENTYP][, …]
) RETURNS (<parametername> DATENTYP][, …])
LANGUAGE plpgsql AS $$
DECLARE <variablenname> DATENTYP [DEFAULT WERT];
BEGIN
   <anweisung>;
      …
 RETURN <rückgabewerte>;
END; $$
```

Unterschied zur Prozedur?

- **Rückgabewert**
- **RETURN QUERY SELECT … möglich**
- **Aufruf mit SELECT <Pr-Name>**

- Für Rückgabe einer Tabelle: RETURNS TABLE (…)
- Aufruf der Funktion:

```
SELECT <functionname> (<parameterliste>);
```

5 Trigger



Trigger
Spezielle „Stored Procedure“, die durch bestimmte Datenbankoperationen/Ereignisse ausgelöst wird.

Ereignisse

- DML: DELETE, INSERT, UPDATE
- DDL: CREATE, ALTER, DROP
- DB-Operationen: SERVERERROR, LOGON, LOGOFF, STARTUP, SHUTDOWN

5 Trigger

- Umsetzung von Business Rules und komplexeren Konsistenzregeln
- Intelligenz in der Datenbank, nicht im Programm
  - Auslösung nur an einer Stelle
  - Reduktion der Netzlast
- Komplexere Fehlersuche

5 Trigger – PostgreSQL

```
CREATE OR REPLACE TRIGGER <TriggerName>
[BEFORE | AFTER]
[ INSERT | UPDATE | DELETE ]
ON <Tabelle>
FOR EACH ROW EXECUTE FUNCTION <FunktionsName>();
```

```
CREATE OR REPLACE FUNCTION <FunktionsName> ()
RETURNS TRIGGER LANGUAGE PLPGSQL AS $$
DECLARE
    <variable> integer = 5;
BEGIN
<Anweisungen>
RETURN NULL;
END;
$$;
```

```
-- Test auf Triggeroperation
IF (TG_OP = 'DELETE') THEN
   OLD…
ELSIF (TG_OP = 'UPDATE') THEN
   NEW…
ELSIF (TG_OP = 'INSERT') THEN
   NEW…
END IF;
```

```
DROP TRIGGER <triggername>;
```

https://www.postgresql.org/docs/current/plpgsql-trigger.html

5 Trigger – MariaDB

```
CREATE TRIGGER <triggername>
{BEFORE | AFTER}
{INSERT | UPDATE | DELETE}
ON <tabellenname>
FOR EACH ROW
BEGIN
<anweisung>;
…
END
```

- BEFORE: Ausführung vor der SQL-Anweisung
- AFTER: Ausführung nach der SQL-Anweisung

```
DROP TRIGGER <triggername>;
```

5 Trigger – Zugriff auf Datensatzwerte

- OLD: Alter Wert
- NEW: Neuer Wert

- Zuweisung zu einer Variablen

```
SET @<variablenname> = {NEW | OLD}.<spaltenname>;
```

- Änderung neuer Werte in BEFORE-Trigger möglich

```
SET NEW.<spaltenname> = <ausdruck>;
```

5 Trigger – Verfügbarkeit von Datensatzwerten

UPDATE *Spalte name: von Müller nach Huber*		INSERT *Spalte name: Huber*		DELETE *Spalte name: Müller*	
BEFORE	**AFTER**	**BEFORE**	**AFTER**	**BEFORE**	**AFTER**
Verfügbarkeit des alten Wertes					
OLD.name ***Müller***	–	–	–	**OLD.name** ***Müller***	**OLD.name** ***Müller***
Verfügbarkeit des neuen Wertes					
NEW.name ***Huber***	**NEW.name** ***Huber***	**NEW.name** ***Huber***	**NEW.name** ***Huber***	–	–

5 Trigger – Beispiel PostgreSQL

```
CREATE FUNCTION gastsichern() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO altgaeste (gnr, gname) VALUES (old.gnr, old.name);
  RETURN new;
END; $$
```

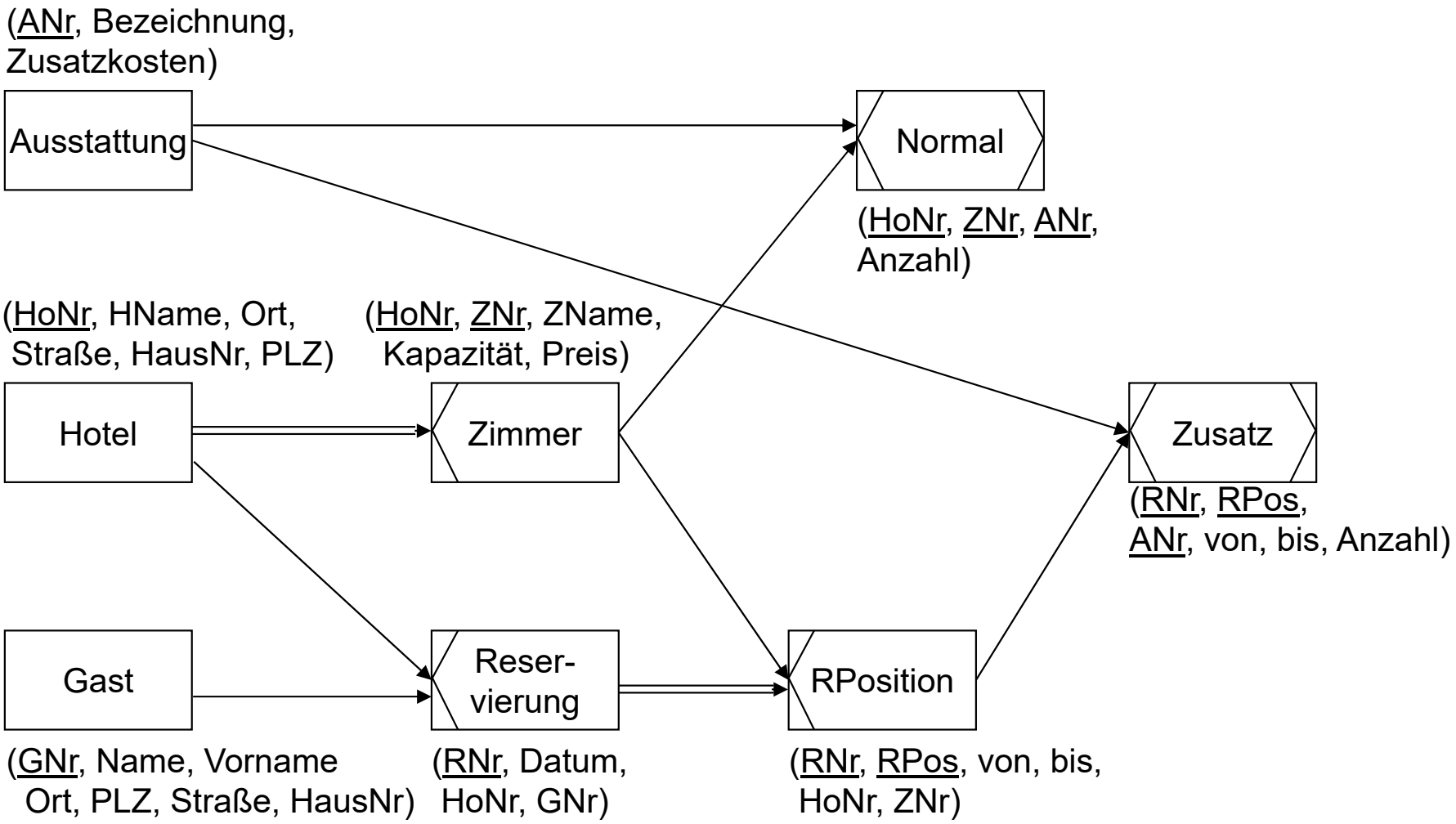
```
CREATE TRIGGER gaestesichern
BEFORE DELETE ON gast
FOR EACH ROW
EXECUTE FUNCTION gastsichern();
```

5 Trigger – Beispiel MySQL

```
CREATE TRIGGER gaestesichern
BEFORE DELETE ON gast FOR EACH ROW
EXECUTE FUNCTION
  INSERT INTO altgaeste (gnr, gname)
                    VALUES (old.gnr, old.name);
```

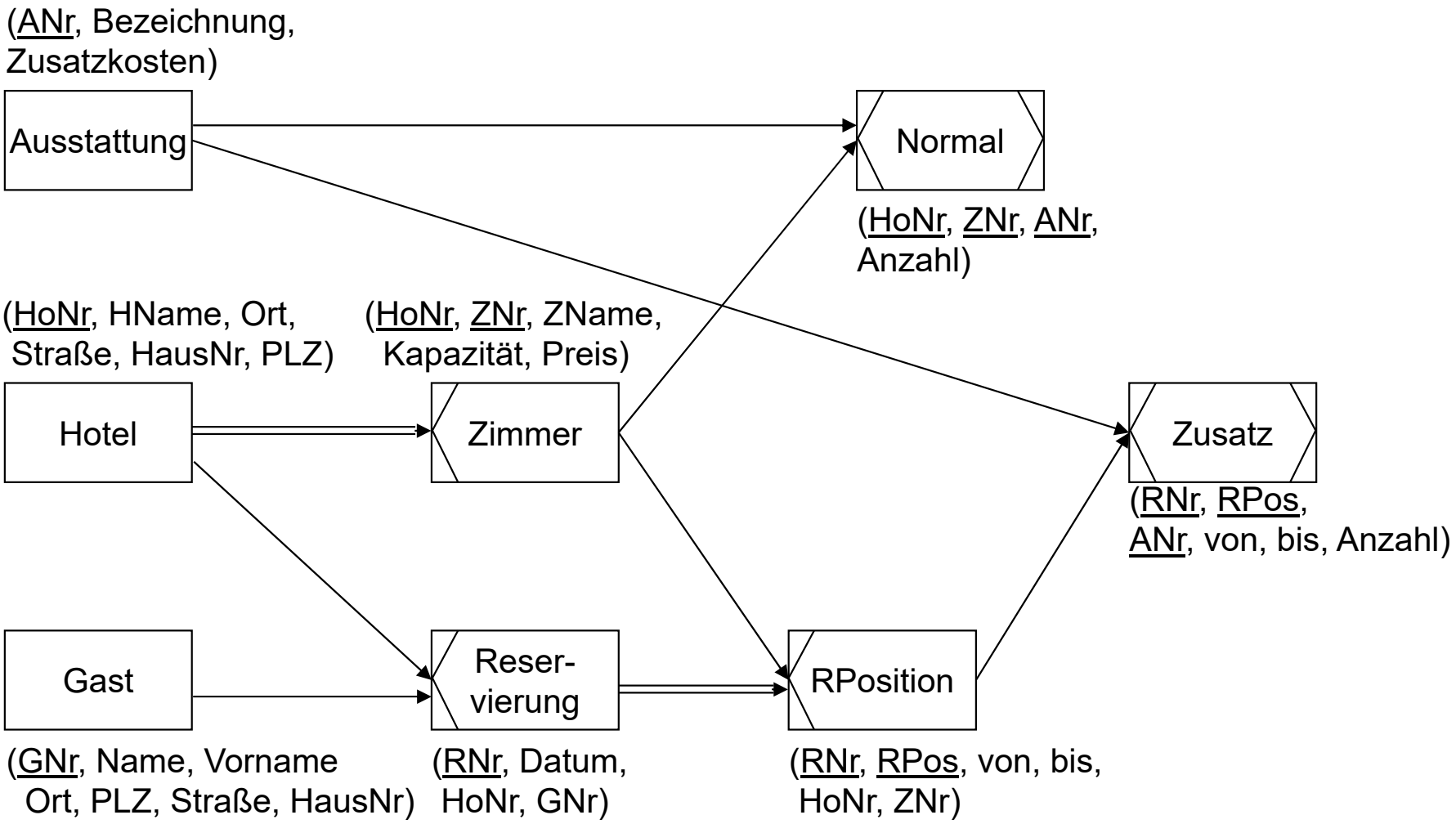
5 Aufgaben – Stored Procedures

- Prozedur zum Anlegen eines Zimmers mit allen Attributen über den Hotelnamen



5 Aufgaben – Stored Procedures

- Prozedur zum Anpassen der Mehrwertsteuer bei Zimmer- und Ausstattungspreisen. Eingabe ist alte und neue MwSt.



5 Aufgaben – Stored Procedures

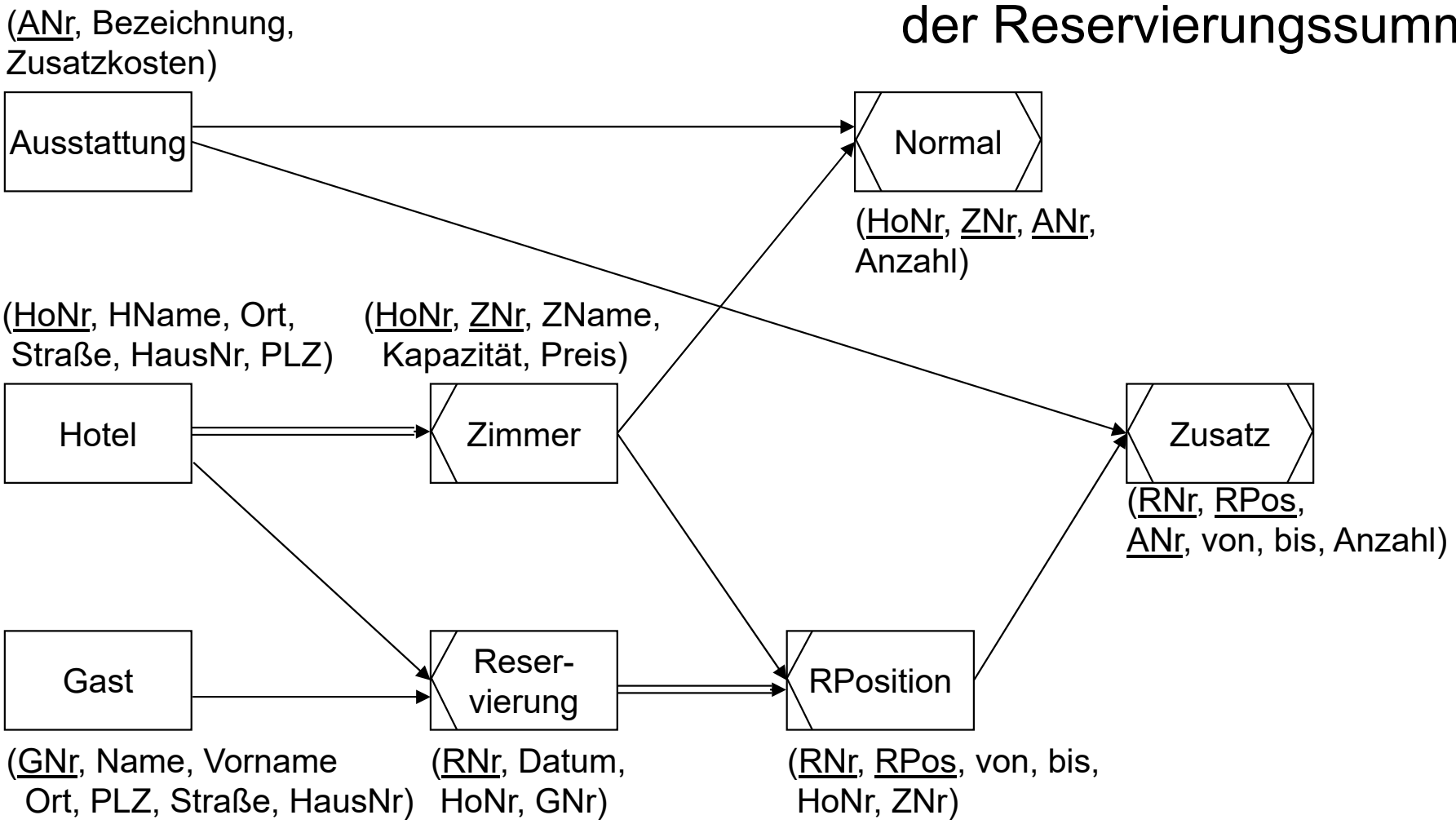
- Prozedur zur Erstellung einer Tabelle, in der eine Maschine linear abgeschrieben wird
  Beispiel:
  - Wert 21.000 Euro
  - Dauer: 7 Jahre

  ⇒Rate: 21.000 Euro / 7 Jahre = 3.000 Euro/Jahr

Jahr	Rate	Restwert
0	0	21.000
1	3.000	18.000
2	3.000	15.000
3	3.000	12.000
4	3.000	9.000
5	3.000	6.000
6	3.000	3.000
7	3.000	0

5 Aufgaben – Trigger

- Verwaltung einer Reservierungssumme in Reservierung nach jeder neuen Rechnungsposition. Vorab Erstellung/Initialisierung der Reservierungssumme.



5 Metadaten

Metadaten (Data Dictionary)
Informationen über die Struktur einer Datenbank sowie der darin enthaltenen Daten

5 Metadaten

- Tabellen
  - Tabellenname
  - Attribute & Typen
  - Constraints
  - Trigger
  - Anzahl Datensätze
  - Indizes
- Views
- Benutzer & Accountinformation

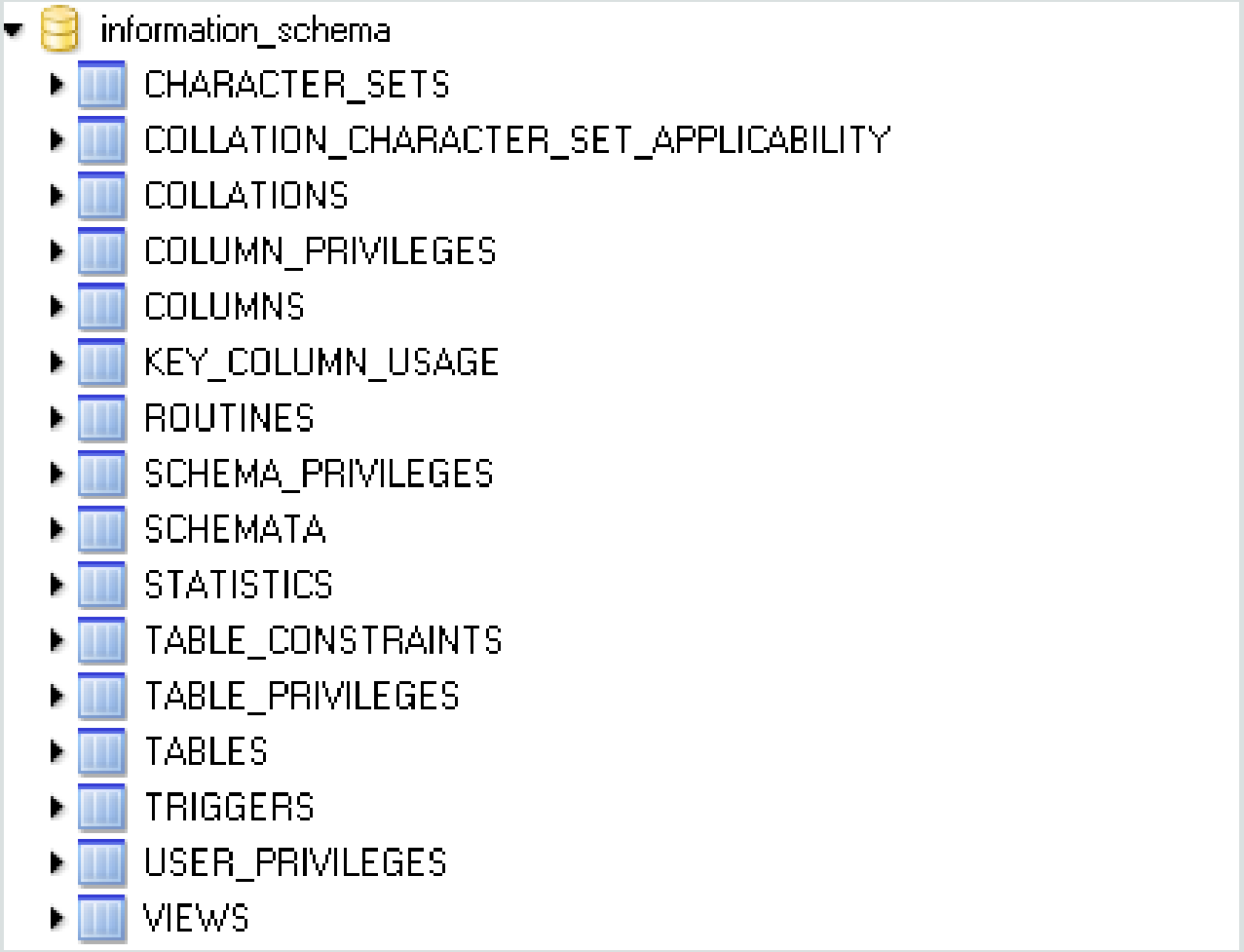
5 Metadaten – Auswahl (Oracle)

- SYS.USER_OBJECTS
  all objects owned by you, where objects include: tables, views, indexes…
- SYS.TAB
  views & tables owned by you
- SYS.USER_TABLES
  tables owned by you
- SYS.USER_VIEWS
  views owned by you
- SYS.ALL_TABLES
  tables that you have permission to access whether your own, or someone else's
- SYS.USER_TAB_COLUMNS
  columns that each table has
- SYS.USER_CONSTRAINTS
  constraints on tables created (owned) by you
- SYS.USER_TRIGGERS
  triggers owned by you
- SYS.USER_CATALOG
  similar to SYS.TAB, with a simpler structure

5 Metadaten – Systematik (Oracle)

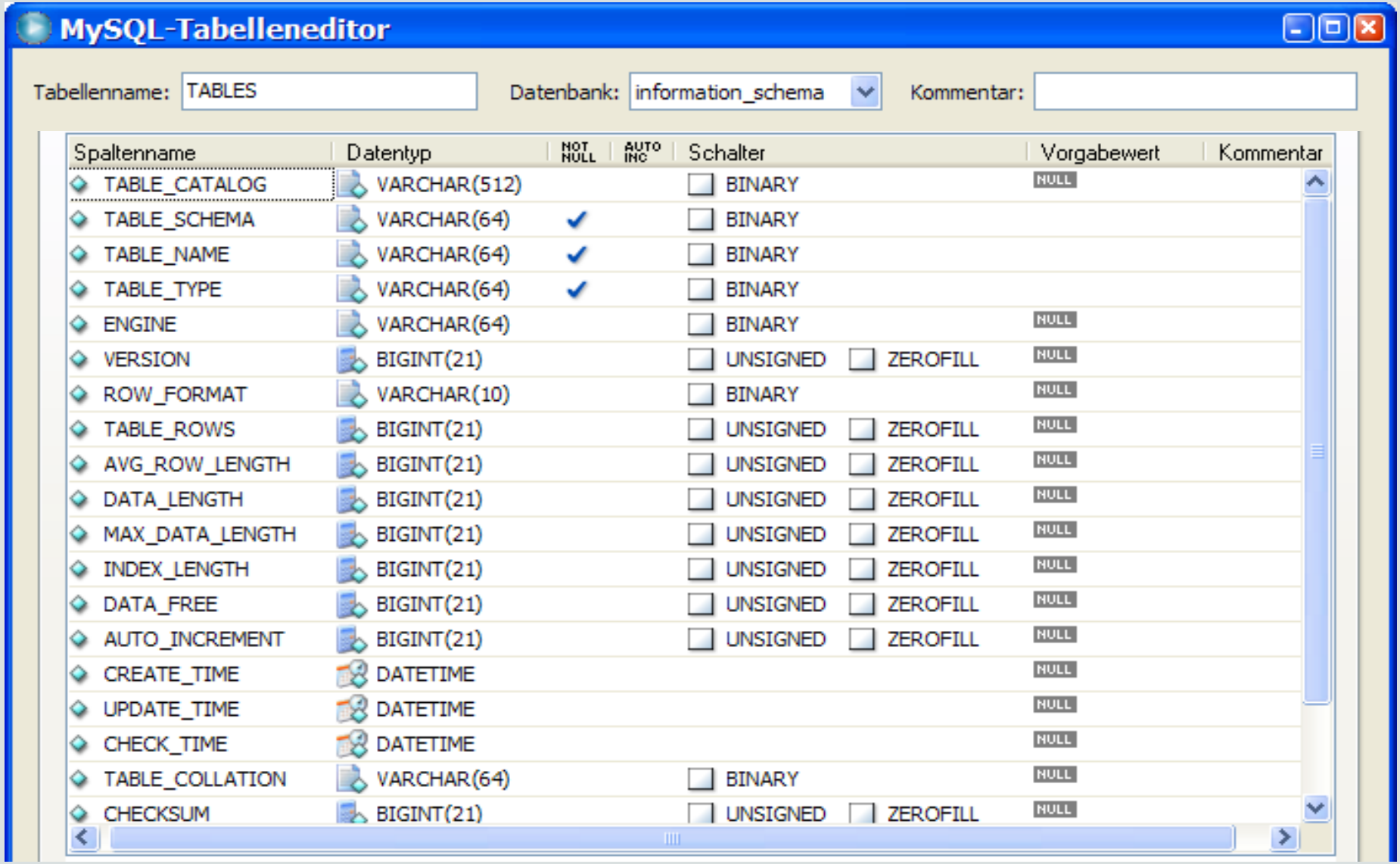
- **USER_**
  Informationen über DB-Objekte, deren Owner des User ist
- **ALL_**
  Informationen über DB-Objekte, auf die ein User Zugriff hat, auch wenn er nicht der Owner ist
- **DBSA_**
  Nur für Administratoren zugängliche Informationen über alle Objekte, unabhängig von Owner-Status und Zugriffsrechten

5 Metadaten – MySQL-Datenbank-Katalog

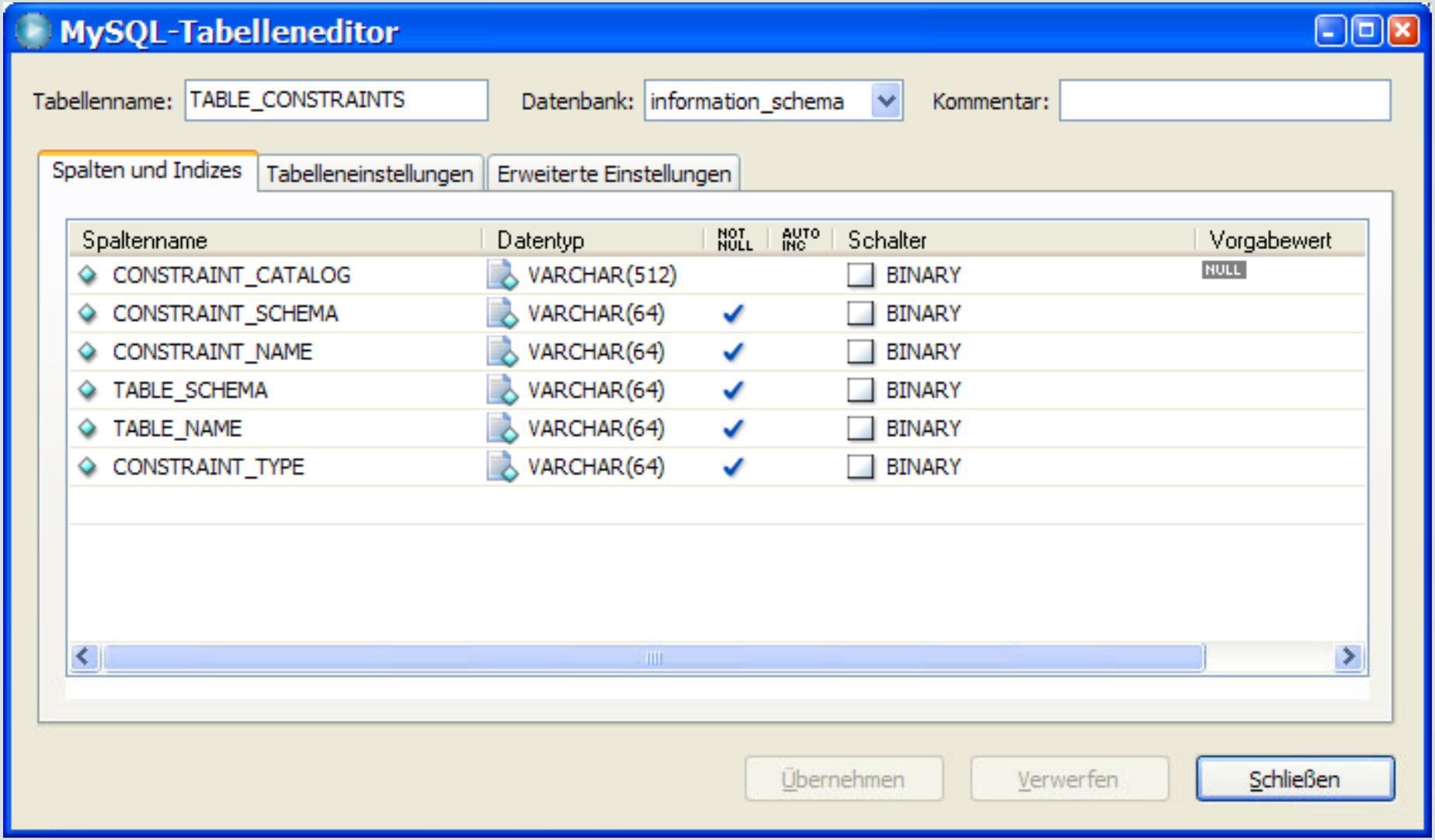


Die Katalogtabelle befinden sich im Schema *information_schema*

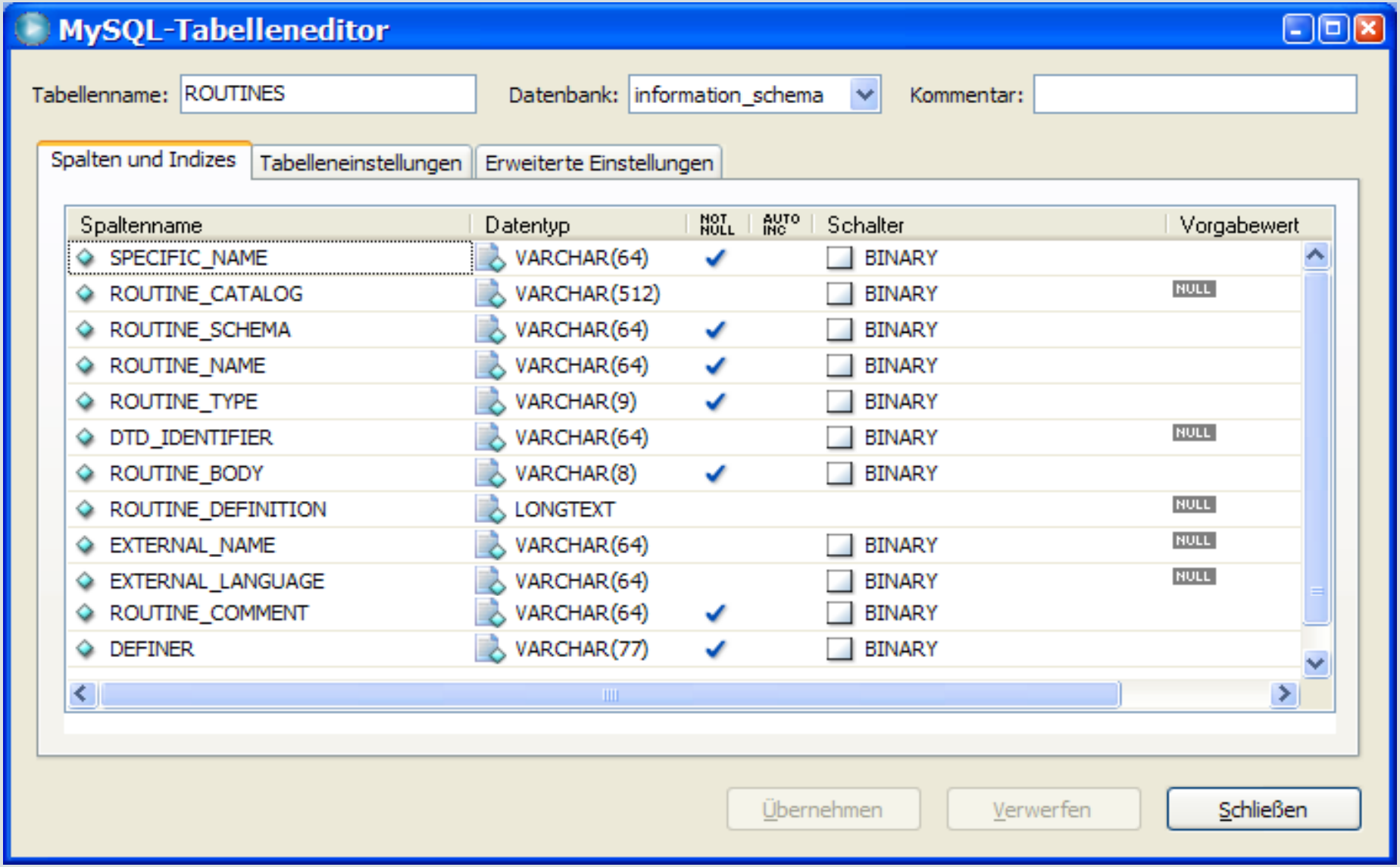
5 Metadaten – MySQL Tables



5 Metadaten – MySQL Table_Constraints



5 Metadaten – MySQL Routines



5 Metadaten – MySQL Triggers

